EXP 1:

QUESTIONS 5 / 5 completed

Classic Binary Sear

CLASSIC BINARY SEARCH

An online dictionary app wants to look up whether a word's rank exists in a sorted list of N ranks as fast as possible. Write a program implementing binary search on a sorted array.
(1) Read N and the sorted array.

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>

int main() {
    int N, target, i;
    int comparisons = 0;
    int foundIndex = -1;

    // Reading N
    if (scanf("%d", &N) != 1) return 0;

    int arr[N];

    // Reading the sorted array
    for (i = 0; i < N; i++) {
        scanf("%d", &arr[i]);
    }

    // Reading the target value
    scanf("%d", &target);

    // Binary Search Implementation
```

INPUT

```
7
2 5 8 12 16 23 38
16
```

EXP 2:

QUESTIONS 5 / 5 completed

First Occurrence Se

FIRST OCCURRENCE SEARCH

A library catalogue has many books with the same rating repeated; the system needs the position of the very first book with a given rating in a sorted list. Write a program using divide and conquer (modified binary search) to find the first occurrence of a target

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>

int findFirstOccurrence(int arr[], int n, int target) {
    int low = 0, high = n - 1;
    int result = -1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == target) {
            result = mid;     // Record the index
            high = mid - 1;   // Keep searching the left side to find the *first* occu
        } else if (arr[mid] < target) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return result;
}
```

INPUT

```
8
2 4 4 4 5 7 7 9
4
```

EXP 3:

QUESTIONS 5 / 5 completed

Rotated Array Sear

ROTATED ARRAY SEARCH

A circular parking garage's ticket numbers were sorted but then the array got rotated when the system restarted at a different point. Write a program that searches for a target value in a rotated sorted array using a divide-and-conquer approach,

PROGRAM EDITOR C Run Save

```c
#include <stdio.h>

int rotatedSearch(int arr[], int low, int high, int target) {
    if (low > high) {
        return -1;
    }

    int mid = low + (high - low) / 2;

    if (arr[mid] == target) {
        return mid;
    }

    // Check if left half is sorted
    if (arr[low] <= arr[mid]) {
        if (arr[low] <= target && target < arr[mid]) {
            return rotatedSearch(arr, low, mid - 1, target);
        } else {
            return rotatedSearch(arr, mid + 1, high, target);
        }
    }
```

INPUT

```
7
4 5 6 7 0 1 2
0
```

EXP 4:

QUESTIONS 5 / 5 completed

Peak Element D&C

PEAK ELEMENT D&C

A stock analysis tool needs to quickly locate any single day where the price was higher than both its neighbouring days (a local peak), without checking every day one by one. Write a program using divide and conquer to find a peak element in an

PROGRAM EDITOR C Run Save

```c
#include <stdio.h>

int findPeakUtil(int arr[], int low, int high, int n) {
    int mid = low + (high - low) / 2;

    if ((mid == 0 || arr[mid - 1] <= arr[mid]) &&
        (mid == n - 1 || arr[mid + 1] <= arr[mid])) {
        return mid;
    }
    else if (mid > 0 && arr[mid - 1] > arr[mid]) {
        return findPeakUtil(arr, low, mid - 1, n);
    }
    else {
        return findPeakUtil(arr, mid + 1, high, n);
    }
}

int findPeak(int arr[], int n) {
    return findPeakUtil(arr, 0, n - 1, n);
}
```

INPUT

```
6
1 3 20 4 1 0
```

EXP 5:

QUESTIONS 5 / 5 completed

Count Occurrences

COUNT OCCURRENCES D&C

A survey system stores sorted response scores and needs to quickly count how many respondents gave an exact target score, without scanning every entry. Write a program that uses binary search (D&C) to find the first and last occurrence of a target

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>

int findFirst(int arr[], int low, int high, int target) {
    if (low > high)
        return -1;

    int mid = low + (high - low) / 2;

    if ((mid == low || arr[mid - 1] < target) && arr[mid] == target)
        return mid;
    else if (arr[mid] < target)
        return findFirst(arr, mid + 1, high, target);
    else
        return findFirst(arr, low, mid - 1, target);
}

int findLast(int arr[], int low, int high, int target) {
    if (low > high)
        return -1;

    int mid = low + (high - low) / 2;
```

INPUT

```
8
12222345
2
```